

Reinforcement Learning

Prediction and Control with Approximation

Sutton/Barto* Chapter 9-11

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction,
2nd edition, MIT Press, Cambridge, MA, 2018



Online Material



Topics of this Course

- Introduction to reinforcement learning
- Markov decision processes
- Part I: Tabular Methods
 - Dynamic programming
 - Monte Carlo methods
 - Temporal-difference learning
 - Multi-step bootstrapping
 - Planning and learning with tabular methods
- **Part II: Approximate Solution Methods**
 - **Prediction and Control using Approximation**
 - Eligibility Traces
 - Policy Gradient Methods
- Part III: Modern RL Methods
 - Deep Reinforcement Learning
 - Current Applications

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\mathbf{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\mathbf{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bold)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X , i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\operatorname{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or q_*

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r s, a)$	probability of transition to state s' and receiving reward r , from state s taking action a .
$p(s' s, a)$	probability of transition to state s' from state s taking action a .
$r(s, a)$	expected immediate reward from state s after action a .
$r(s, a, s')$	expected immediate reward from state s to s' with action a .
$\pi(a s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Temporal Difference Learning

U_t	target for estimate at time t
δ_t	TD error

Approximate Solution Methods

Motivation and Definition

Motivation

Issue: Tabular methods do not scale for large state spaces.

- a. **Space complexity:** Memory needed for storing the value function with $O(|\mathcal{S}|)$ entries as a table.
- b. **No Generalization:** Many states will never be encountered during learning and therefore cannot be predicted.

Idea: Try to find an approximate value function for policy π :

$$\hat{v}(s, \mathbf{w}) \approx v_\pi \quad \text{with } \mathbf{w} \in \mathbb{R}^d$$

Requirements:

- The function needs to have a **manageable number of parameters**, the weight vector $\mathbf{w}$. I.e., $d \ll |\mathcal{S}|$.
- The function can only depend on **features of state s** .
- The function needs to **generalize**, so states with similar features get a similar value.
Note: Approximation functions do this automatically, since all states share the weight vector and will be affected by changes in the weights.

Prediction: Value Function Approximation

Approach: Use **function approximation** (supervised learning).

- We need **training data** in the form of

$$s \mapsto u,$$

where the target u is a “backed-up value” representing the desired value.

Some options for training data $S_t \mapsto U$:

- MC: $S_t \mapsto G_t$
- TD(0): $S_t \mapsto R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$
- DP: $S_t \mapsto \mathbb{E}_\pi[R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)]$

Note: The subscript t for $\mathbf{w}$ indicates when the update happens during learning. The subscript is often dropped (we only keep track of the current $\mathbf{w}$)

Prediction Objective ($\overline{VE}$)

- **Learning algorithm:** Update the weights $\mathbf{w}$ to reduce the error for $\hat{v}(s, \mathbf{w}) \approx v_\pi$
- **Note on generalization:** Any change of $\mathbf{w}$ will also affect the prediction for other states and thus reduce or increase their error!
- **Prediction error** for state s : $|v_\pi(s) - \hat{v}(s, \mathbf{w})|$

- To calculate the error over all states we need to define how much we care about each state. We use a state distribution function

$$\mu(s) \geq 0; \quad \sum_s \mu(s) = 1$$

- $\mu(s)$ is typically the fraction of time spent in s when following π . We obtain data by following a policy π so samples already comes from this distribution.
 - For continuous tasks, $\mu(s)$ is a stationary distribution called the on-policy distribution.
 - For episodic tasks a near stationary distribution only exists for ergodic* MDPs with large horizons.

[See Code](#)

Definition: Mean square value error

$$\overline{VE} \stackrel{\text{def}}{=} \sum_{s \in \mathcal{S}} \mu(s) [v_\pi - \hat{v}(s, \mathbf{w})]^2$$

Remember: Squared errors useful mathematical and statistical properties that make learning and optimization easier and more effective.

*Ergodic: The Markov process when following π is irreducible (all states are reachable) and aperiodic (does not get stuck).

Learning Goal

- Find the parameter vector $\mathbf{w}^*$ that leads to the smallest error (the global optimum):

$$\overline{VE}(\mathbf{w}^*) \leq \overline{VE}(\mathbf{w}) \quad \forall \mathbf{w}$$

- **Local optima:** Often, we will only find a local optimum where the equation above only holds for $\mathbf{w}$ in a small neighborhood around $\mathbf{w}^*$.

Note: Policies converge typically way before the value function converges so finding the global optimal value function may not be necessary.

- **Convergence:** Convergence to the global or a local optimum depends on the used approximation and the learning method. We will discuss this for each method.
- **Bias:** Different approximation methods have different bias (are not able to represent the true value function).

Gradient Methods for Prediction

Gradient Learning Method

Setup

- For a given policy π , learn an approximation $\hat{v}(s, \mathbf{w}) \approx v_\pi$
- We use $\mathbf{w} \stackrel{\text{def}}{=} (w_1, w_2, \dots, w_d)^\top$
- Choose $\hat{v}(s, \mathbf{w})$ as a **differentiable function** of $\mathbf{w}$ for all $s \in \mathcal{S}$

Learning

- We sample following π , and observe at each discrete time step $t = 0, 1, 2, 3, \dots$ a new example

$$S_t \mapsto v_\pi(S_t).$$

For now, we assume that we can assess the correct $v_\pi(S_t)$

- **Update $\mathbf{w}$** to reduce $\overline{VE}$. I.e., make $\hat{v}(s, \mathbf{w})$ more similar to v_π .
Note that the samples automatically follow the distribution μ for π .

$$\overline{VE} \stackrel{\text{def}}{=} \sum_{s \in \mathcal{S}} \mu(s) [v_\pi - \hat{v}(s, \mathbf{w})]^2$$

SGD: Stochastic-Gradient Descend

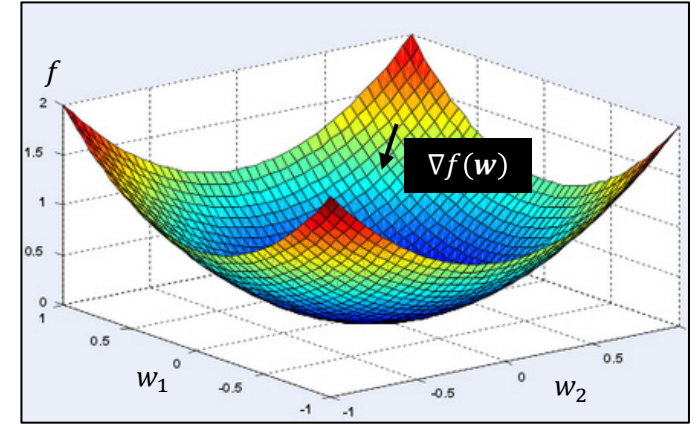
Minimize $\overline{VE} \stackrel{\text{def}}{=} \sum_{s \in \mathcal{S}} \mu(s) [v_\pi - \hat{v}(s, \mathbf{w})]^2$ by moving $\mathbf{w}$ in the direction $\nabla \overline{VE}$.

Definition of the gradient: $\nabla f(\mathbf{w}) = \left(\frac{\partial f(\mathbf{w})}{\partial w_1}, \frac{\partial f(\mathbf{w})}{\partial w_2}, \dots, \frac{\partial f(\mathbf{w})}{\partial w_d} \right)^\top$

Update for one observation: $S_t \mapsto v_\pi(S_t)$

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t - \frac{1}{2} \alpha \nabla [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)]^2 \\ &= \mathbf{w}_t + \alpha [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t) \end{aligned}$$

- **Important property of SGD:** Only depends on the gradient of the loss function. Does not need the gradient of the unknown value function!
- **Stochastic:** because update is on a single (randomly chosen) example for S_t . This is why there is no sum in the update.
- **Step size α :** Since we update towards a sampled value and updates of $\mathbf{w}$ for one state will change the value for other states as well, we use many small updates (small value of α).



Applying the chain rule* shows that we only need the gradient of the approximation function.

Notes: the sign changes and $\frac{1}{2}$ was chosen for convenience.

***Chain rule** (of differentiation)

$$h(x) = z(y(x))$$

is equivalent to

$$h'(x) = z'(y(x)) y'(x)$$

Step Size for SGD

- SGD converges to a **local optimum** if the **stochastic approximation criterion** (Robbins–Monro conditions) is met:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

$\alpha_n(a)$... step size used after the n -th selection of action a

- **Popular choices for α :**

- Tabular MC uses $\alpha_n(a) = \frac{1}{n}$ which leads to sample averaging. **Not appropriate for TD or approximation!**
- Constant $\alpha = 1/\tau$ means a tabular estimate would approach to the mean target after τ experiences + keeps learning.

- **How to choose α in practice**

- Typical values: 0.1, 0.01, 0.001
- If learning diverges: reduce α by a factor of 10
- If learning is too slow: increase α by a factor of 2 to 5.
- Plot learning curve.

A constant does not satisfy the stochastic approx. criterion.

Using a Target Estimate for SGD

Issue: We have assumed that we know the actual value of $v_\pi(S_t)$ for the update

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

but that is not the case in RL where we want to learn v_π .

Solution: We can use any target estimate of the expected return U_t instead

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha[U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

SGD converges to a local optimum as long as U_t is an **unbiased estimate**. I.e.,

$$\mathbb{E}(U_t | S_t = s) = v_\pi(s).$$

Examples:

- The Gradient Monte Carlo method uses the unbiased MC estimate $U_t = G_t$

Gradient Monte Carlo Algorithm

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π

 Loop for each step of episode, $t = 0, 1, \dots, T - 1$:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

Remember from MC:

$$G_t = \sum_{k=0}^T \gamma^k R_{t+k+1}$$

Bootstrapping and Semi-Gradient Methods

Issue:

- To get the true gradient requires, the target has to be unbiased, fixed and only the prediction can depend on $\mathbf{w}$.

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha[U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

Gradient Monte Carlo Algorithm: Works since $U_t = G_t$ is the result of only sampled rewards which are independent of $\mathbf{w}$.

Bootstrapping methods (e.g., n -step TD): break this requirement because U_t depends on the bootstrap estimate $\hat{v}(S_{t+n}, \mathbf{w})$ and thus the current value of $\mathbf{w}$. This does not result in a true gradient, and convergence cannot be guaranteed.

Semi-Gradient Methods:

- If we use the approach with a biased target estimate, then this is called a semi-gradient method.
- Advantages:
 - Are usually faster learners.
 - Often still converge (especially for linear approximators).
 - TD can learn online.

Semi-gradient TD(0)

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose $A \sim \pi(\cdot|S)$

 Take action A , observe R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$

$S \leftarrow S'$

 until S is terminal

U_t is a bootstrapped estimate and biased because it uses $\mathbf{w}$.
Convergence is not guaranteed!

Linear Approximation Methods

The approximation function $\hat{v}(s, \mathbf{w})$

Linear Methods

- Linear approximation of the state value by the inner product

$$\hat{v}(s, \mathbf{w}) \stackrel{\text{def}}{=} \mathbf{w}^\top \mathbf{x}(s) = \sum_{i=1}^d w_i x_i(s)$$

$\mathbf{x}(s)$... feature vector with $x_i: \mathcal{S} \rightarrow \mathbb{R}$ (called basis function)

- **SGD gradient:** $\nabla \hat{v}(s, \mathbf{w}) = \left(\frac{\partial \hat{v}(s, \mathbf{w})}{\partial w_1}, \frac{\partial \hat{v}(s, \mathbf{w})}{\partial w_2}, \dots, \frac{\partial \hat{v}(s, \mathbf{w})}{\partial w_d} \right)^\top = (x_1, x_2, \dots, x_d)^\top = \mathbf{x}(s)^\top$
- **SGD update:** $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \mathbf{x}(S_t)$

- **Advantages:**

- Simple math.
- Has only one optimum and is **guaranteed to converge** to the **global minimum** of $\overline{VE}$ if α is reduced following the stochastic approximation condition.
- Convergence guarantee also holds for **bootstrapping** methods like Semi-gradient TD(0)!

These are extremely strong guarantees!

Feature Vector

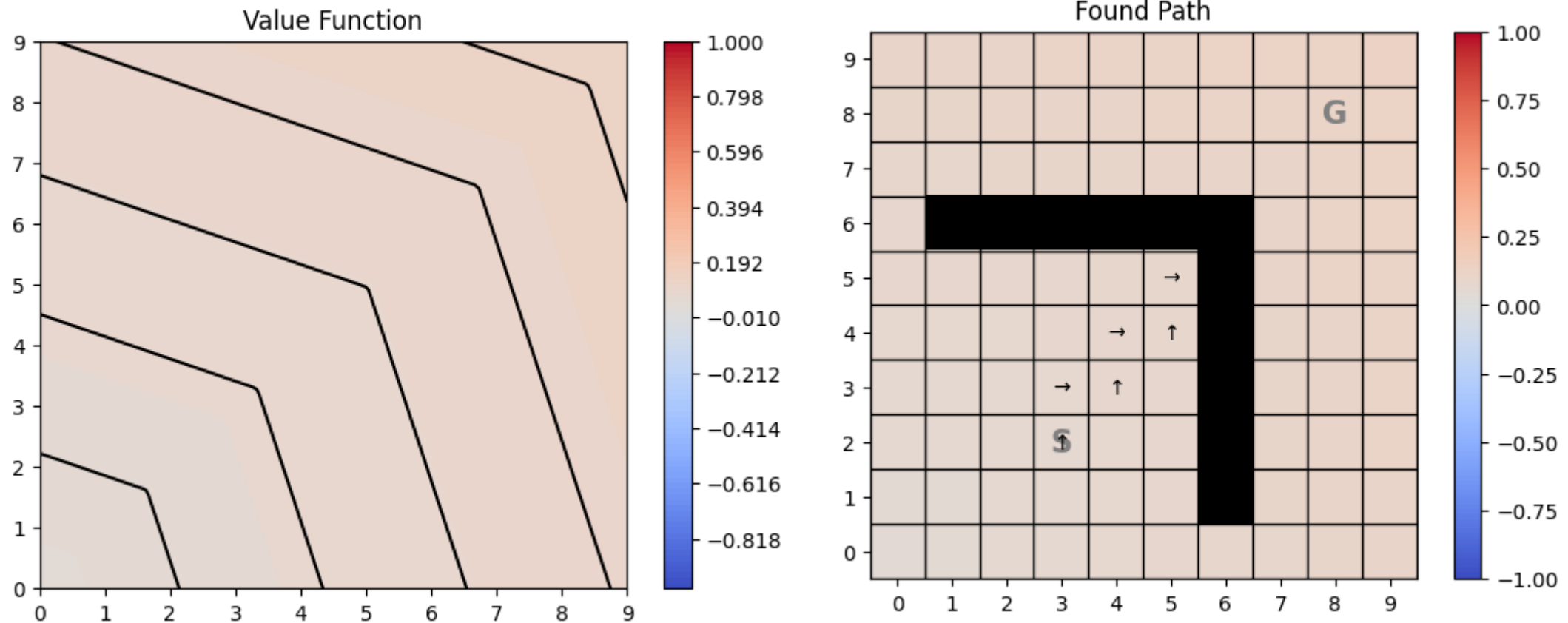
- **Issue:** How do we construct the feature vector $\mathbf{x}(s)$ for a state?
- **Initial idea:** If the factored state description consists of several numbers (e.g., x and y coordinates), then we can directly use them as the feature vector $\mathbf{x}(s) = (s_x, s_y)$.

This leads to **linear interpolation** learned as a **linear regression**.

- **Problems:**
 - The value function may not depend linearly on the variables used in the state description.
 - Linear regression cannot represent interactions between the variables.

Example: L-Maze with Linear Features

Semi-gradient Sarsa (see Control later) with linear features cannot solve the L-Maze



Feature Construction

- **Idea:** Use a more flexible, non-linear transform of the state description to construct state features.
- We will focus on the most popular feature construction methods that work well for RL:
 - **Fourier Basis:** strong baseline; very efficient; strong guarantees.
 - **Tile Coding:** coarse coding using tiles, local generalization, very computationally efficient.
- Other methods
 - One-Hot Features: exact representation; no generalization (equivalent to tabular RL)
 - Coarse Coding using spherical reception fields
 - Polynomials Features: smooth global generalization.
 - Radial Basis Functions: smooth local generalization

Fourier Basis

- A **Fourier series** (Fourier transform) represents a periodic function as a weighted sum of sine and cosine basis functions of increasing frequencies.
- State features are not periodic, so we consider only a single period, chosen so that we only need to use cosine basis functions.
- **On dimensional case:** state is represented by a single number $s \in [0,1]$

$$x_i(s) = \cos(i\pi s)$$

π is 180° in radians

with $i = 0, \dots, n$ where i is the frequency and n is the order of the Fourier basis function

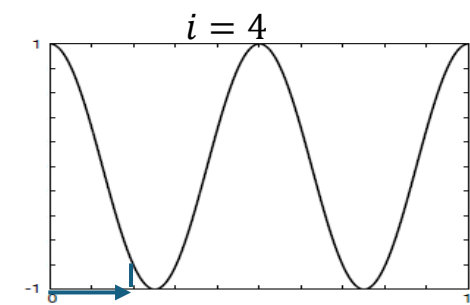
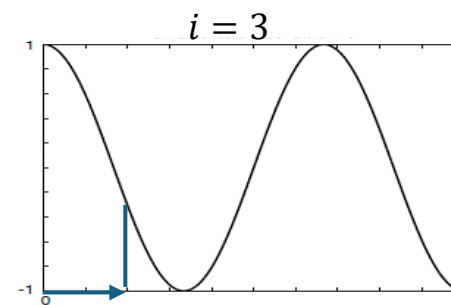
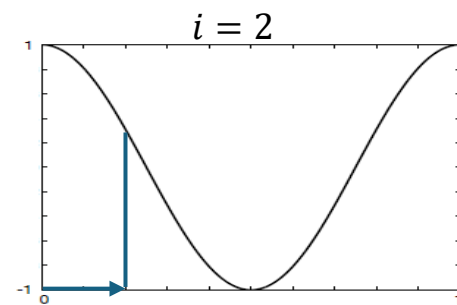
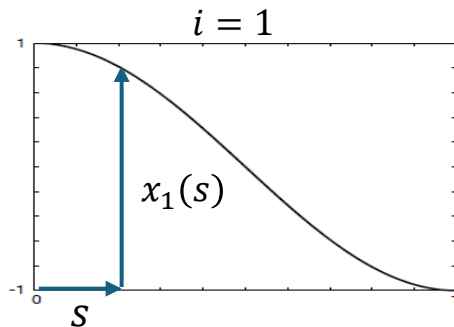
- Strong theoretical foundation: Fourier series can approximate any function.

Example: Fourier Transform



Source: Wikipedia

Example: Order-4 Fourier cosine basis creates 4 features plus the intercept for $i = 0$ for linear approximation.



$$x(.2) = (1, .9, .65, .35, .1)$$

Multi-dimensional Fourier Basis

- State is represented by k numbers: $\mathbf{s} = (s_1, s_2, \dots, s_k)$

$$x_i(\mathbf{s}) = \cos(\pi \mathbf{s}^\top \mathbf{c}^i)$$

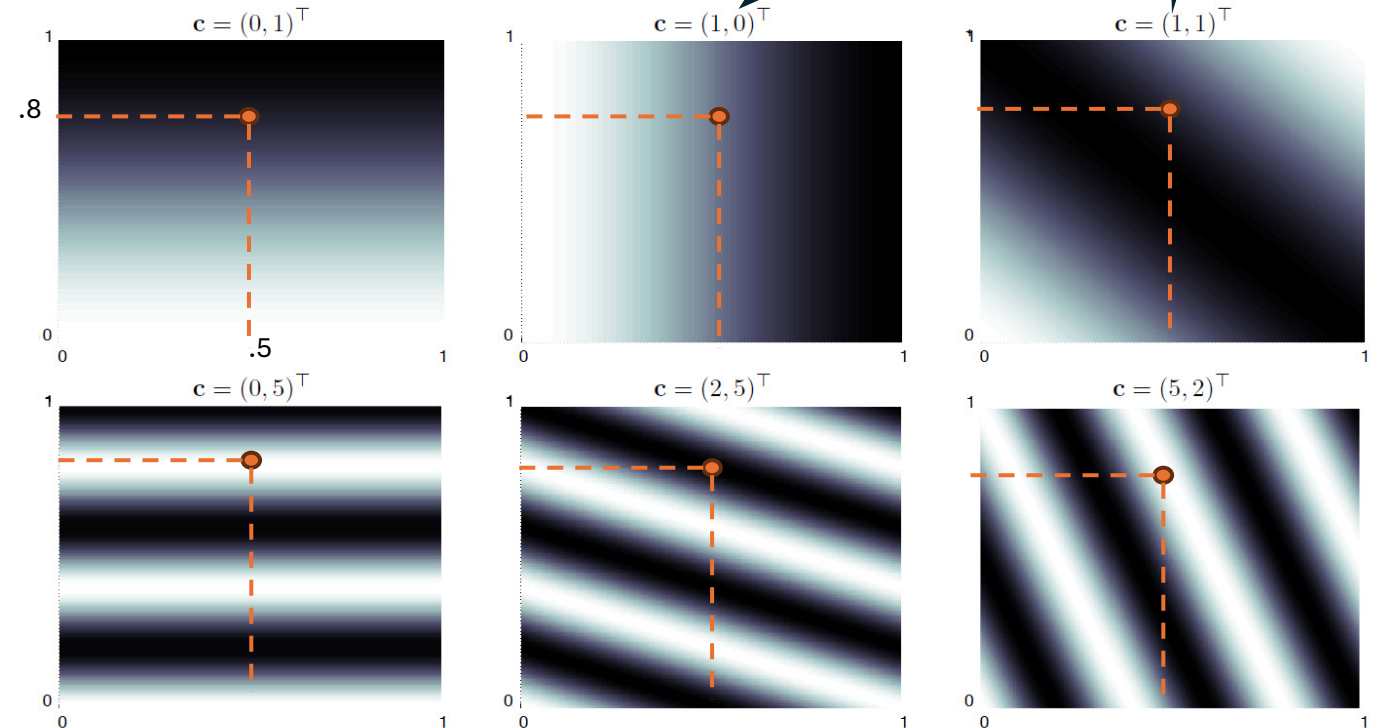
where $\mathbf{c}^i = (c_1^i, \dots, c_k^i)$
with $c_j^i \in \{0, \dots, n\}$ for $j = 1, \dots, k$ and $i = 1, \dots, (n+1)^k$

Example: 2-dimensional Fourier cosine features for 6 different $\mathbf{c}$.

$$\mathbf{s} = (.5, .8)$$

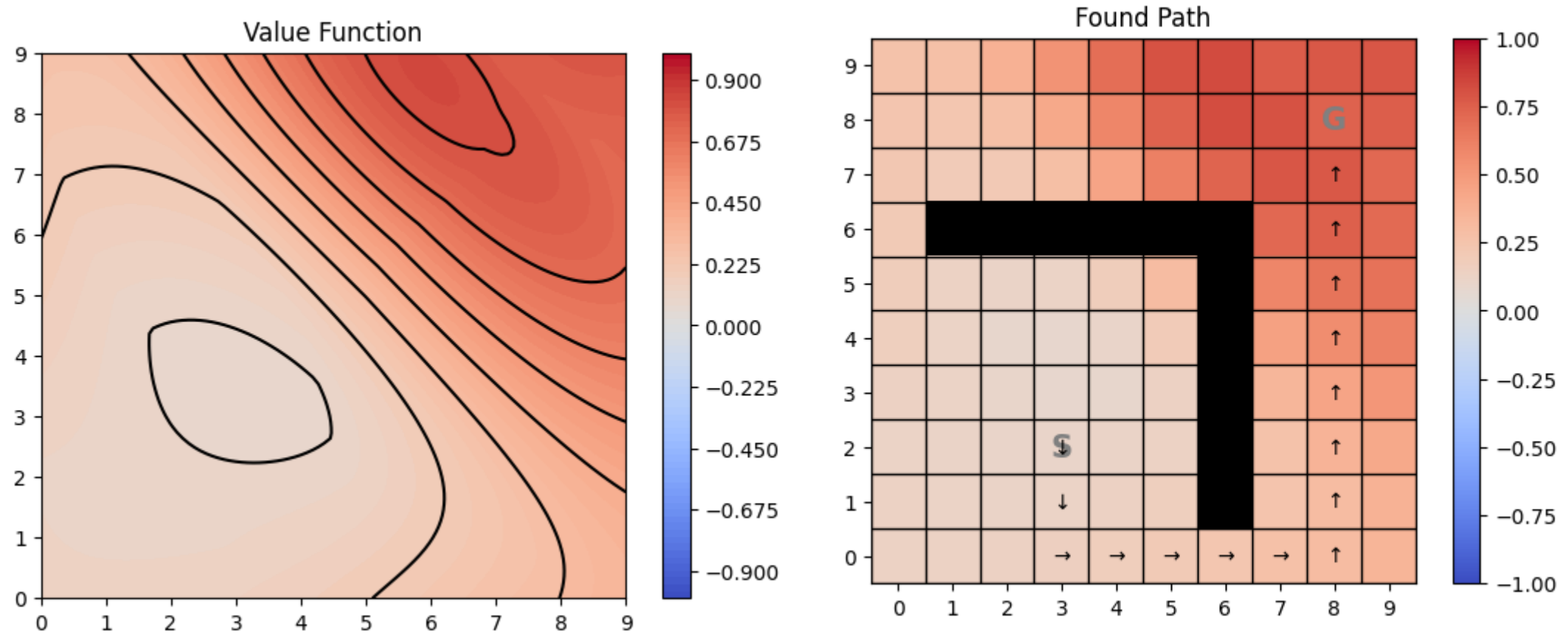
$$\begin{aligned}x_1(\mathbf{s}) &= \cos(\pi (.5, .8)^\top (0, 1)) = -.8 \\x_2(\mathbf{s}) &= \cos(\pi (.5, .8)^\top (1, 0)) = 0 \\x_3(\mathbf{s}) &= \cos(\pi (.5, .8)^\top (1, 1)) = -.6 \\x_4(\mathbf{s}) &= \cos(\pi (.5, .8)^\top (0, 5)) = 1 \\x_5(\mathbf{s}) &= \cos(\pi (.5, .8)^\top (2, 5)) = -1 \\x_6(\mathbf{s}) &= \cos(\pi (.5, .8)^\top (5, 2)) = 1\end{aligned}$$

1. Features are a combination of both state components, representing interactions.
2. Select only the best features.
3. We can use a different α for each feature.



Example: L-Maze with Order-2 Fourier Features

Semi-gradient Sarsa with order-2 Fourier Features can solve the L-Maze

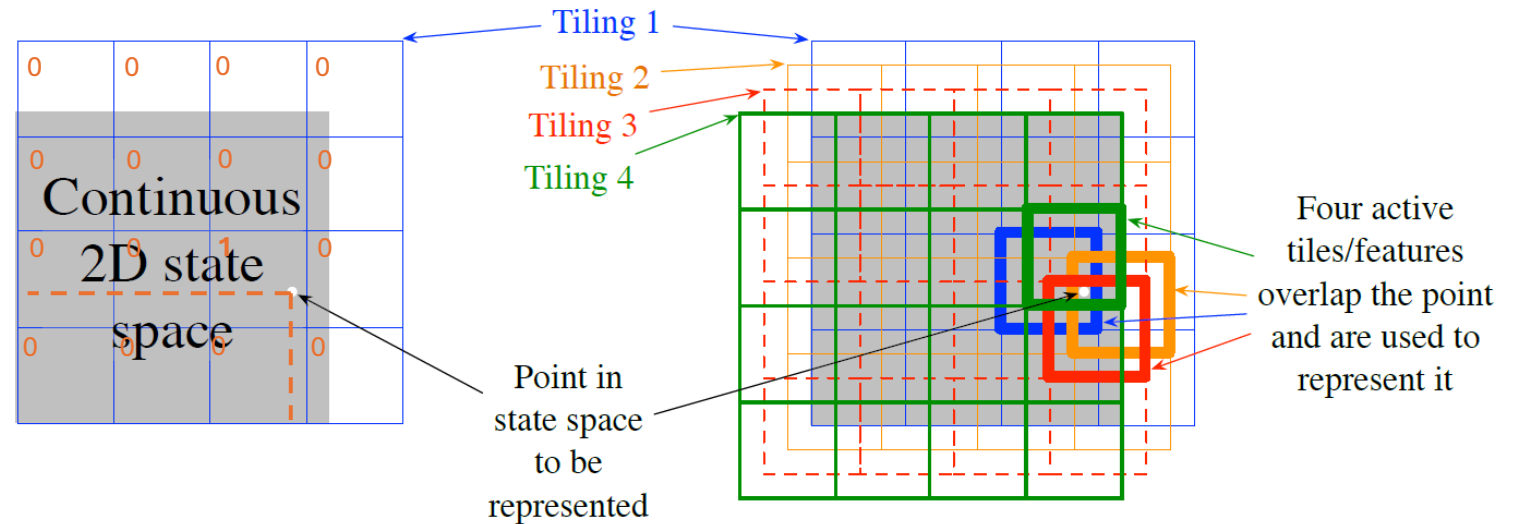


Tile Coding

- Partition the space using several **overlapping grids**.
- Represent the state by the “**active**” tiles in the grids.
- **Similar states will share active tiles.** Leads to a local generalization.
- α is typically chosen to be small since a change will also affect neighboring states.
Popular: $\alpha = \frac{1}{10 \text{ \#tilings}}$

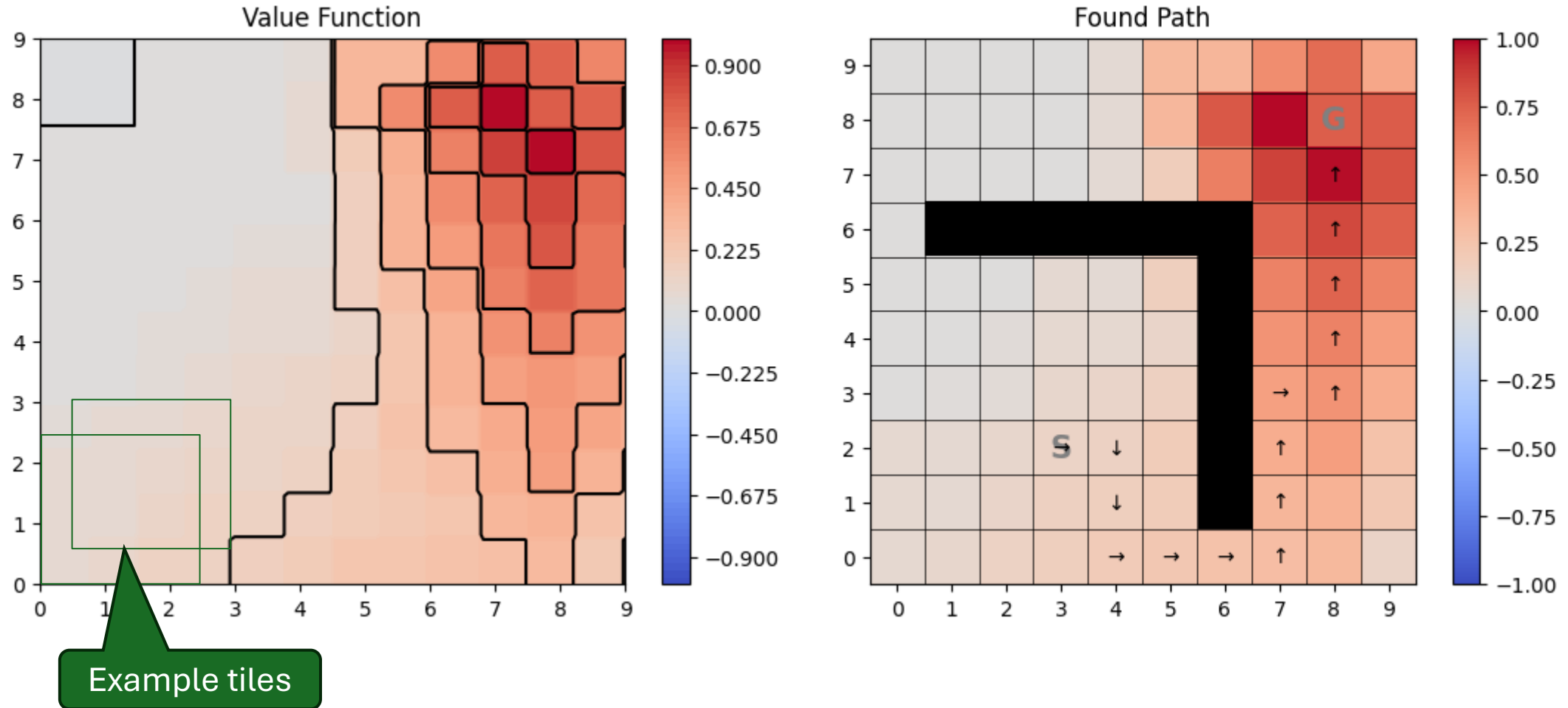
Example:

- State has two components.
- 4×4 overlapping tilings give 64 squares.
- State features: 60 0s and only the four active tiles have a 1



Example: L-Maze with Tile Coding

Semi-gradient Sarsa with Tile Coding (4 tilings, 4 tiles per dimension) can solve the L-Maze



Partially Observable Environments

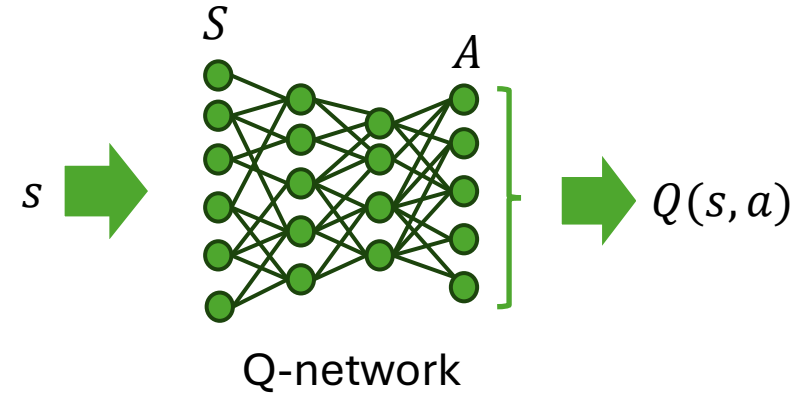
- Observations may be incomplete or noisy.
- Standard model: **POMDP** with belief states
 - A key feature is that it learns **hidden states** depending on the history of actions and observations.
 - **Impractical** due to the size of the belief state space.
- Practical approach: RL
 1. Build a **compressed representation** with features from current observations and the observation history
 2. **Use a (non) linear approximator** on those features
 3. Learn a value function or policy from the compressed representation.
- **Example:** Maze solver where the x and y coordinates are noisy. Using the change in coordinates for the last few steps helps.

Non-Linear Function Approximation

Artificial Neural Networks

Approximation with ANNs

- Uses simple networks with a single hidden layer or deep architectures to approximate the value function.
- Methods:
 - Recurrent networks
 - Convolution networks
 - Transformers
 - Regularization, dropout, weight sharing
 - Deep residual learning...
- Networks are trained like linear models using SGD. For deep networks this uses backpropagation.
- ANNs are non-linear universal approximators so they can learn any value function.
- Issues due to nonlinearity:
 - Instability
 - Convergence is not guaranteed.
- More in Deep Reinforcement Learning (DRL).



On-Policy Control with Approximation

Semi-Gradient Sarsa

Control Problem

- Estimate a parametric approximation for value-action function.

$$\hat{q}(s, a, \mathbf{w}) \approx q_*(s, a)$$

- **Training data:** $S_t, A_t \mapsto U_t$
- **Update:** $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{q}(S_t, A_t, \mathbf{w}_t)] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$
 - U_t can be any approximation:
 - MC: $U_t = G_t$
 - 1-step TD: $U_t = R_{t+1} + \gamma \hat{q}(S_t, A_t, \mathbf{w}_t)$
 - ...

We need state-action features!

- **Policy improvement:** update policy with greedy action

$$A_{t+1}^* = \operatorname{argmax}_a \hat{q}(S_{t+1}, a, \mathbf{w}_t)$$

- For on-policy control, use a soft approximation to the greedy policy (e.g., ϵ -greedy).

State-Action Features

- We need to learn to predict $\hat{q}(s, a, \mathbf{w})$ which should have a different value for each action.
- Geramifard et al. (2013)* suggest to have different state-feature weights for each action.
- **Example:** A state described by x and y and three actions, the weight vector would be

$$\mathbf{w} = (w_{\text{intercept}}, w_{x,\text{action1}}, w_{y,\text{action1}}, w_{x,\text{action2}}, w_{y,\text{action2}}, w_{x,\text{action3}}, w_{y,\text{action3}})$$

- In the prediction/update, only the intercept and the weights belonging to the action are used.

*Alborz Geramifard, Stefanie Tellex, and Thomas J. Walsh. A Tutorial on Linear Function Approximators for Dynamic Programming and Reinforcement Learning, Foundations and Trends in Machine Learning, Vol. 6, No. 4 (2013) 375–454

Episodic Semi-gradient Sarsa

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameter: step size α

Initialize value-function weights $\mathbf{w}$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose $A \sim \pi(\cdot|S)$

 Take action A , observe R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})]$

$S \leftarrow S'$

 until S is terminal

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

$S, A \leftarrow$ initial state and action of episode (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 If S' is terminal:

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

 Go to next episode

 Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε -greedy)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

$S \leftarrow S'$

$A \leftarrow A'$

Considerations for Continuing Tasks

Discounting vs. Average Reward

- Discounted rewards (γ -returns) are used for tabular methods because returns for each state are separately identified and averaged.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

- With approximation, returns are shared (generalized) between states and it is often better to focus on the long-run **average reward** per time step.

$$r(\pi) = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T R_t$$

Differential Returns

- It is often helpful to use differences between the immediate rewards and the average reward given policy π .

$$G_t = \sum_{k=1}^{\infty} (R_{t+k+1} - r(\pi))$$

- This leads to a differential value function.



What You should Know

- Approximate the value function using parametrized functions:

$$\hat{v}(s, \mathbf{w}) \text{ or } \hat{q}(s, a, \mathbf{w})$$

- For linear approximation, we need to create **features** that take non-linearities and the interaction of state components into account. Popular are:
 - Fourier basis features
 - Tile coding
- Use **SGD or semi-gradient descent** to learn $\mathbf{w}$ from samples by minimizing $\overline{VE}$. The samples already follow the on-policy distribution $\mu(s)$.
- Issue: **Convergence**
 - Biased target estimates?
 - Linear vs. non-linear approximation.
 - choice of α .